

Resilience & Rate Limiting Cheatsheet

Lessons: [36 — Resilience Patterns](#) · [68 — Rate Limiting & Throttling](#)

Examples: [36](#) · [68](#)

Covers: timeout, retry, circuit breaker, bulkhead, the five limiter algorithms, `x/time/rate`, distributed limits

Legend: `[*]` = real API that the lessons have not covered yet

THE RESILIENCE TOOLKIT

timeout	bound EVERY outbound call; the first line of defence
retry	only for transient failures, only for idempotent calls
backoff + jitter	exponential, randomized – never retry in lockstep
circuit breaker	stop calling a service that is clearly down
bulkhead	isolate resources so one dependency can't drain the pool
rate limit / load shed	protect yourself from callers
fallback	a cached/default answer beats an error page
graceful degradation	serve less, but serve
(they compose in this order: shed -> break -> bulkhead -> retry -> timeout -> call)	

TIMEOUTS

```
ctx, cancel := context.WithTimeout(ctx, 2*time.Second)
defer cancel()
client := &http.Client{Timeout: 10 * time.Second}    the whole-request cap
srv.ReadTimeout / WriteTimeout / IdleTimeout        the server side
budget, not a constant      a 3s request cannot afford three 2s calls
propagate the deadline      ctx carries it into the DB call and the next hop
deadline < caller's deadline each hop shrinks the budget it passes on
(a call with no timeout is a goroutine leak waiting for a network partition)
```

RETRY

retry ONLY on	timeouts, 5xx, connection refused, codes.Unavailable
NEVER on	400/401/403/404/422 – the answer will not change
NEVER on non-idempotent	unless you carry an idempotency key
backoff	$d = \text{base} * 2^{\text{attempt}}$, capped at a max
jitter	$d = \text{rand}(0, d)$ – the thing that prevents thundering herds
cap attempts	3 is usually right; unbounded retry is an outage amplifier
respect Retry-After	if the server told you when, believe it
budget the retries	"at most 10% extra load" beats "3 attempts each"
retry storm	every layer retrying $3x = 27x$ load on the bottom service – retry at ONE layer only

CIRCUIT BREAKER

closed	normal; count failures
open	fail FAST without calling; the dependency gets to recover
half-open	after a cooldown, let a few probes through
probe succeeds -> closed ; probe fails -> open again	
trip on	failure RATE over a window, not a raw count
minimum volume	don't trip on 2 of 2 requests
what it protects	your latency and your goroutines, not the dependency
per-dependency	one breaker per downstream service, not one global
sony/gobreaker	[*] the common Go implementation
combine with a fallback	open circuit -> cached value, default, or a clean 503

BULKHEAD & LOAD SHEDDING

bulkhead	a fixed budget per dependency so it can't take everything
sem := make(chan struct{}, 10)	the semaphore form
sem <- struct{}{} ; defer func(){ <-sem }()	
semaphore.NewWeighted(n)	[*] the x/sync version
separate pools	a slow reporting query shouldn't starve the login path
load shedding	reject early when you are already saturated
if inflight > max { 503 + Retry-After }	
queue depth as the signal	latency-based shedding beats a fixed concurrency cap
priority shedding	drop the bulk export before dropping the checkout
(the goal is to fail 1% of requests fast instead of 100% of them slowly)	

THE FIVE RATE-LIMIT ALGORITHMS

fixed window	count per clock window; simplest
the flaw	2x burst at the boundary (n at 0:59, n at 1:00)
sliding window log	timestamps in a sorted set; exact, memory-hungry
sliding window counter	weighted blend of the current and previous window; the usual production compromise
token bucket	tokens refill at rate r, capacity b; SPENDS bursts the standard choice – allows bursts, bounds the average
leaky bucket	a queue drained at a constant rate; SMOOTHS output good for protecting a fragile downstream
choose	token bucket for APIs; leaky bucket for egress; sliding counter when the window must be exact

x/time/rate (the standard Go limiter)

```
lim := rate.NewLimiter(rate.Limit(10), 20)    10 events/sec, burst of 20
lim.Allow()                                  take one token now, or false – the middleware form
lim.AllowN(t, n)                             n tokens at a given time
lim.Wait(ctx)                                block until a token is free, or ctx dies
lim.WaitN(ctx, n)                            the same for n
r := lim.Reserve()                           always succeeds; tells you how long to wait
    r.OK() / r.Delay() / r.Cancel()
lim.SetLimit(r) / lim.SetBurst(b)             retune at runtime
rate.Every(100*time.Millisecond)             a Limit expressed as an interval
rate.Inf                                      [*] an unlimited limiter
```

TRAP: Wait returns its OWN error, NOT wrapped context.DeadlineExceeded – errors.Is(err, context.DeadlineExceeded) is false. Check ctx.Err() yourself.

TRAP: Reservation.Cancel() only refunds the token when Delay() > 0. A reservation that was immediately available is already spent.

NOTE: NewLimiter(r, b) permits b immediately, then r per second on average.

PER-KEY LIMITERS

```
map[string]*rate.Limiter    per user / per IP / per tenant / per API key
mu sync.Mutex               or sync.Map – this map is written on every request
THE LEAK                   every key ever seen stays forever
    fix 1                   an LRU with a bounded size
    fix 2                   a lastSeen timestamp + a sweeper goroutine
    fix 3                   a TTL cache
tiered plans                free/pro/enterprise -> different (r, b) per tier
composing limits            per-key AND global: check both, charge ONCE
                             (Reserve both, cancel the other if either denies)
the key matters             IP behind a proxy is the proxy; use the identity you trust
```

DISTRIBUTED RATE LIMITING

```
the problem                N replicas, one shared budget
Redis INCR + EXPIRE        fixed window; simple, has the boundary burst
Lua token bucket            atomic read-modify-write in one round trip
GCRA                        [*] the "leaky bucket as a virtual timestamp" algorithm;
                             one value per key, no windows, smooth
sorted set of timestamps   exact sliding window, most memory
local + global              a local limiter for the fast path, Redis for the truth

failure mode – decide it deliberately:
    fail-open               Redis is down -> allow (availability first)
    fail-closed             Redis is down -> deny (protection first)
    fail-local              fall back to a per-replica limiter (usually the best)
```

THE 429 CONTRACT

429 Too Many Requests	the status
Retry-After: 30	seconds, or an HTTP date – always send it
RateLimit-Limit / -Remaining / -Reset	the draft standard headers
X-RateLimit-*	the older de-facto names
document the limits	a limit clients can't discover will be hit
metrics	allowed/denied counters, by key class and by rule
never 500 on a limit	it's an expected outcome, not an error

TRAPS & MEMORIZE

retry without jitter	synchronized retries hammer the recovering service
retrying at every layer	multiplicative load; retry at one layer only
retrying POSTs	duplicate side effects without an idempotency key
no timeout on the client	http.Get with no Timeout can hang forever
one global circuit breaker	one bad dependency opens the circuit for all of them
breaker that never half-opens	the dependency recovered an hour ago
unbounded per-key map	the memory leak in every rate limiter tutorial
limiting after the work	count it BEFORE the expensive part
rate limit != concurrency limit	10/s of 30s requests is 300 in flight
Wait() with no ctx deadline	an unbounded queue of blocked goroutines
fixed window on a boundary	2x the intended burst, every window
fail-open by accident	"Redis was down so we let everything through"